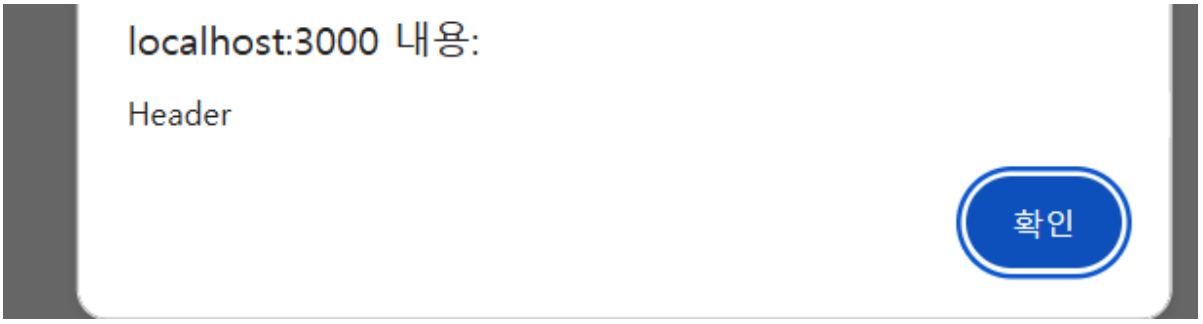
React

Event

```
function Header(props) {
  return <header>
    <h1><a href="/" onClick ={function(event){
      event.preventDefault();
      props.onChangeMode();
    }}>{props.title}</a></h1>
  </header>
}

function App() {
  return (
    <div>

      <Header title="WEB" onChangeMode={function(){
        alert('Header');

      }}></Header>

    </div>
```

HeaderにWEBを押すとpreventDefaultでページの移動防ぐ。

props.onChangeMode()を実行してalert('Header')が実行する。



state

Reactには入力と出力がある。

入力がpropである。porpを通じて入力されたデータを関数が処理して returnをする。

propとともにstateがある。

propは外部、stateは内部の違いがある。

```
const [mode, setMode] = useState('WELCOME');
const [id, setId] = useState(null);
```

```
}
return (
  <div>
    <Header title="WEB" onChangeMode={()=>{
      setMode('WELCOME');
    }}></Header>
    <Nav topics={topics} onChangeMode={(_id)=>{
      setMode('READ');
      setId(_id);
    }}></Nav>
```

ボタンを押してちを変更する時ただ変更すればならない。

stateを使って変更するべきである。

WEB

1. html
2. css
3. javascript

css

css is ...

このように変更される。

Create

```
function Create(props){
  return <article>
    <h2>Create</h2>
    <form onSubmit={event=>{
      event.preventDefault();
      const title = event.target.title.value;
      const body = event.target.body.value;
      props.onCreate(title, body);
    }}>
      <p><input type="text" name="title" placeholder="title"/></p>
      <p><textarea name="body" placeholder="body"></textarea></p>
      <p><input type="submit" value="Create"></input></p>
    </form>
  </article>
```

```
} else if(mode === 'CREATE'){
  content = <Create onCreate={(_title, _body)=>{
```

新しい投稿を作るためにCreate関数を作る。

formを使用してtitleとbodyをに送る。

```
const [nextId, setNextId] = useState(4);
const [topics, setTopics] = useState([
```

```
} else if(mode === 'CREATE'){
  content = <Create onCreate={(_title, _body)=>{
    const newTopic = {id:nextId, title:_title, body:_body}
    const newTopics = [...topics]
    newTopics.push(newTopic);
    setTopics(newTopics);
    setMode('READ');
    setId(nextId);
    setNextId(nextId+1);
  }}></Create>
```

画面に追加することを見せるためにstateを設定してnewTopicをpushすると新しい投稿がでる。

```
<a href="/create" onClick={event=>{
  event.preventDefault();
  setMode('CREATE');
}}>Create</a>
```

このようになる。

Update

```
else if(mode === 'READ'){
  let title, body = null;
  for(let i=0; i<topics.length; i++){
    console.log(topics[i].id, id);
    if(topics[i].id === id){
      title = topics[i].title;
      body = topics[i].body;
    }
  }
  content = <Article title={title} body={body}></Article>
  contextControl = <a href="/update+id">Update</a>
```

```
{contextControl}
```

READにUpdateをcontextControlとして定義する。

```
function Update(props){
  return <article>
    <h2>Update</h2>
    <form onSubmit={event=>{
      event.preventDefault();
      const title = event.target.title.value;
      const body = event.target.body.value;
      props.onUpdate(title, body);
    }}>
      <p><input type="text" name="title" placeholder="title" value={props.title}/></p>
      <p><textarea name="body" placeholder="body" value={props.body}></textarea></p>
      <p><input type="submit" value="Update"></input></p>
    </form>
  </article>
}
```

```
} else if(mode === 'UPDATE'){
  let title, body = null;
  for(let i=0; i<topics.length; i++){
    if(topics[i].id === id){
      title = topics[i].title;
      body = topics[i].body;
    }
  }

  content = <Update title={title} body={body} onUpdate={(title, body)=>{
  }}></Update>
```

3. javascript

Update

javascript

javascript is ...

Update

このようにして修訂しても文字を入力することができない。

原因はinputのvalueをprops.title, porps.bodyにしたからである。

```
const [title, setTitle] = useState(props.title);
const [body, setBody] = useState(props.body);

return <article>
  <h2>Update</h2>
  <form onSubmit={event=>{
    event.preventDefault();
    const title = event.target.title.value;
    const body = event.target.body.value;
    props.onUpdate(title, body);
  }}>
    <p><input type="text" name="title" placeholder="title" value={title} onChange={event=>{
      setTitle(event.target.value);
    }}/></p>
    <p><textarea name="body" placeholder="body" value={body} onChange={event=>{
      setBody(event.target.value);
    }}></textarea></p>
```

そのため、stateを使用する。

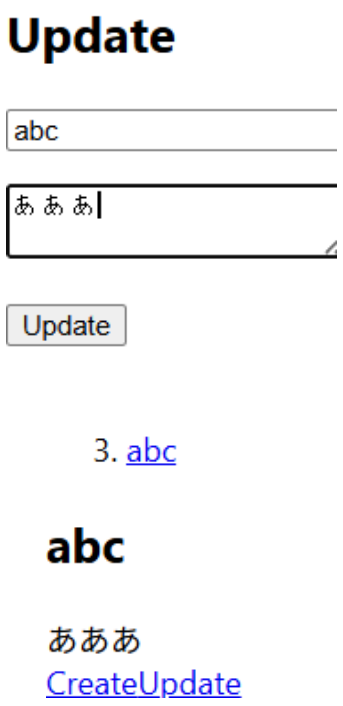
既存の値がvalueに渡された時点でpropsからstateに乗り換える。

値が変更される時にその変更された値をstate に保存し、そのstateの値をinputにフィードバックする必要がある。

```
content = <Update title={title} body={body} onUpdate={(title, body)=>{
  console.log(title, body);
  const newTopics = [...topics]
  const updatedTopic = {id:id, title:title, body:body}
  for(let i=0; i<newTopics.length; i++){
    if(newTopics[i].id === id){
      newTopics[i] = updatedTopic;
      break;
    }
  }
  setTopics(newTopics);
  setMode('READ');
}}></Update>
```

そして、topicsを複製してidをさがす。

その後setTopicsでせっていしたTopicを保存する。



Delete

```
else if(mode === 'UPDATE'){
let title, body = null;
for(let i=0; i<topics.length; i++){
  if(topics[i].id === id){
    title = topics[i].title;
    body = topics[i].body;
  }
}
content = <Update title={title} body={body} onUpdate={(title, body)=>{
  console.log(title, body);
  const newTopics = [...topics]
  const updatedTopic = {id:id, title:title, body:body}
  for(let i=0; i<newTopics.length; i++){
    if(newTopics[i].id === id){
      newTopics[i] = updatedTopic;
      break;
    }
  }
  setTopics(newTopics);
  setMode('READ');
}}></Update>
```

削除はUPDATEの仲を修正する。

選択した項目を除くて新しいTopic配列をつくる。

1. html
2. css
3. javascript

html

html is ...

Create

Update

Delete

1. css
2. javascrip

Welcom

Hello, WEB
Create